

Verifica di teoria – Linguaggi di programmazione ad oggetti (Java e UML)

1. Introduzione alla programmazione ad oggetti

La programmazione orientata agli oggetti (OOP) è un modello di programmazione che rappresenta un sistema come un insieme di oggetti che collaborano tra loro.

Ogni oggetto contiene dati (detti *attributi*) e funzioni (dette *metodi*) e può essere visto come un'entità che possiede caratteristiche e comportamenti propri.

L'OOP permette di creare programmi modulari, riutilizzabili e facili da mantenere, rendendo più semplice gestire progetti di grandi dimensioni.

Rispetto alla programmazione procedurale, che è centrata sulle funzioni, quella a oggetti si concentra sulle entità del problema, rendendo il codice più vicino alla realtà e più organizzato.

2. Concetti fondamentali dell'OOP

I principi principali della programmazione orientata agli oggetti sono quattro:

Astrazione – Consiste nel rappresentare solo gli aspetti essenziali di un oggetto, ignorando i dettagli superflui.

Incapsulamento – Protegge i dati interni di un oggetto: gli attributi non sono accessibili direttamente, ma solo tramite metodi pubblici (*getter* e *setter*).

Ereditarietà – Permette di creare nuove classi (sottoclassi) che ereditano attributi e metodi da una classe esistente (superclasse). Questo favorisce il riuso del codice.

Polimorfismo – Permette di usare lo stesso metodo in classi diverse, ottenendo comportamenti differenti a seconda dell'oggetto che lo invoca.

Questi concetti rendono l'OOP uno strumento potente e flessibile per sviluppare software complesso in modo chiaro e organizzato.

3. Classi, oggetti, attributi e metodi

Una classe è il modello che descrive gli attributi e i metodi comuni a un gruppo di oggetti.

Gli attributi rappresentano le proprietà (ad esempio nome, età, colore), mentre i metodi definiscono le azioni che l'oggetto può eseguire.

Un costruttore è un metodo speciale usato per inizializzare gli oggetti quando vengono creati.

Un oggetto, invece, è un'istanza concreta di una classe, con valori specifici per i suoi attributi.

Esempio in Java:

```
class Studente {  
    String nome;  
    void saluta() { System.out.println("Ciao!"); }  
}
```

Qui la classe `Studente` contiene un attributo e un metodo. Quando si crea un oggetto di tipo `Studente`, questo eredita la struttura definita nella classe.

4. OOD e OOP

Nel metodo di sviluppo orientato agli oggetti si distinguono due fasi principali: Object Oriented Design (OOD) e Object Oriented Programming (OOP).

L'**OOD** riguarda la progettazione del sistema: si analizza il problema, si individuano le classi necessarie e si stabiliscono le relazioni tra di esse.

L'**OOP**, invece, è la realizzazione pratica del progetto, cioè la scrittura del codice basato sul modello realizzato nella fase di design.

In breve: l'OOD serve a progettare, l'OOP serve a programmare.

5. UML – Unified Modeling Language

L'UML (Unified Modeling Language) è un linguaggio grafico usato per rappresentare la struttura e il comportamento di un sistema orientato agli oggetti.

Serve per visualizzare in modo chiaro le classi, gli oggetti e le relazioni tra di essi, prima di scrivere il codice.

Il diagramma più importante è il diagramma delle classi, che mostra per ogni classe:

- il nome della classe,
- gli attributi,
- i metodi,
- e le relazioni con altre classi (come ereditarietà o composizione).

Esempio concettuale:

Una classe Auto può avere attributi come marca e anno, e metodi come accendi().

Una classe Elettrica può estendere Auto aggiungendo un attributo batteria.

Questo si rappresenta graficamente nel diagramma UML.

6. Il linguaggio Java

Java è un linguaggio orientato agli oggetti sviluppato da Sun Microsystems e oggi di proprietà Oracle.

È progettato per essere portabile, sicuro e indipendente dalla piattaforma.

Il suo motto è *"Write once, run anywhere"*, cioè un programma scritto una volta può essere eseguito su qualsiasi sistema operativo grazie alla Java Virtual Machine (JVM).

Vantaggi di Java

- È multiplatforma: lo stesso programma funziona su sistemi diversi.
- Ha un Garbage Collector che libera automaticamente la memoria non più usata.
- È sicuro e impedisce accessi diretti alla memoria.
- Include una libreria standard molto ampia.

Svantaggi

- Le prestazioni sono inferiori rispetto a linguaggi compilati come C o C++.
- Il codice è più lungo da scrivere, poiché la sintassi è più formale.

7. La Java Virtual Machine (JVM)

La JVM (Java Virtual Machine) è la componente che consente di eseguire programmi Java su qualsiasi piattaforma.

Quando un file .java viene compilato, il compilatore produce un file .class contenente bytecode, cioè istruzioni indipendenti dal sistema operativo.

La JVM traduce il bytecode in istruzioni comprensibili dal computer su cui il programma viene eseguito.

All'interno della JVM ci sono diverse aree di memoria:

- Registri: tengono traccia delle istruzioni da eseguire.
- Stack: contiene i metodi in esecuzione e le variabili locali.
- Heap: è l'area dove vengono creati dinamicamente gli oggetti.
- Area dei metodi: memorizza le informazioni sulle classi e sui metodi caricati.

Grazie a questa struttura, Java risulta portabile e gestisce automaticamente la memoria in modo efficiente.

8. Creazione degli oggetti nel Main

Il metodo main è il punto di ingresso del programma Java.

All'interno di esso si possono creare oggetti usando la parola chiave new, che riserva memoria, richiama il costruttore e restituisce un riferimento all'oggetto.

Esempio:

```
Auto a1 = new Auto();
```

```
a1.accendi();
```

Qui viene creato un nuovo oggetto Auto e viene chiamato il suo metodo accendi().